

传输层、端点与 TCP

从主机交付到进程间有状态通信

408 · NET-06

1 本册定位

本 Topic 回答：IP 只能把 packet 尽力送到 host，怎样进一步把数据交给正确 process，并在需要时维持一条双向、有序、流量受控的 byte-stream connection？

本册拥有 port、socket endpoint、multiplexing/demultiplexing、UDP、TCP connection identity、byte sequence state、connection lifecycle、flow control。

通用 Seq/ACK/Timer/Window 正确性由可靠传输拥有；TCP 拥塞算法由拥塞控制拥有。本册只解释它们怎样接入 TCP 状态机。

2 独立阅读词典：从主机定位到进程定位

术语	本册中的可操作定义	不要混成什么
host / process	host 运行协议栈；process 是使用 socket 的浏览器、DNS 服务等执行实体	process 不是 port 数字
port / socket endpoint	port 是主机内的 16-bit 入口号；endpoint 还包含 local IP 和 protocol	都不是整个 TCP connection
multiplex/demultiplex	发送侧汇聚多个应用；接收侧按标识分回正确 socket	不是交换机按 MAC 转发
datagram / segment	UDP 数据单元保留消息边界；TCP segment 是字节流的一段切片	不是 IP packet 的同义词
byte stream	TCP 向应用呈现的有序字节序列，不保留 write 的消息边界	不是一组天然对应的 segment
rwnd / cwnd	rwnd 是接收端还能收多少；cwnd 是发送端估计网络能承受多少	都不是固定链路带宽
MSS / RTT / RTO	MSS 是单段数据上限；RTT 是往返时间；RTO 是等待确认的阈值	MSS 不是 MTU；RTO 不是 RTT
SYN/ACK/FIN/RST	建连同步、确认、正常关闭和异常复位的控制标志	不是四种应用数据类型

本册后文若说“连接”，默认指一对传输端点及其双方的序号、窗口、计时器和状态；若说“报文段”，默认指这条连接在 IP 层之上的一次 TCP 发送，不表示应用的一次完整写操作。

3 根本问题：IP 地址只能找到机器

一台 host 同时运行浏览器、DNS resolver、邮件客户端和多个 server process。只知道 destination IP 仍无法决定把 payload 交给谁。于是传输层增加 endpoint namespace：

Host-to-host delivery $\xrightarrow{\text{port}}$ Process endpoint delivery

发送侧 multiplexing 把多个应用的数据交给 UDP/TCP；接收侧 demultiplexing 根据 protocol 与 endpoint identifiers 找到相应 socket/connection state。

4 名字与对象

- **port number**: 主机内传输端点的 16-bit 标识空间；
- **socket endpoint**: 至少绑定 local IP、transport protocol、local port 等本地信息的通信端点；
- **TCP connection**: 由一对 endpoint 唯一确定, 常表示为 (src IP, src port, dst IP, dst port)；
- **process**: 使用 socket API 的执行实体, 不等于 port 本身；
- **segment/datagram**: TCP/UDP 交给 IP 的传输层数据单元。

同一 server port 可以同时服务很多 TCP connections, 因为 remote endpoint 不同。port 是命名维度, 不是“一条连接只能占一个服务器端口”的独占资源。

5 UDP: 保留消息边界的最小传输接口

UDP 增加 source port、destination port、length 和 checksum, 然后把一个 application datagram 交给 IP。它不建立 transport connection, 也不维护重传、排序、流控和拥塞窗口状态。

5.1 为什么需要 UDP

朴素方案是所有应用都使用可靠 byte stream。失败在于：

- 有些应用需要保留 message boundary；
- 有些请求很短, 不希望先建立传输连接；
- 有些应用要自己控制超时、重传、顺序或实时丢弃策略；
- DHCP 等 bootstrap 流程需要 datagram/broadcast 交互。

UDP 不是“更快的 TCP”, 而是更少策略的接口。低开销换来的代价是应用必须接受丢失、重复、乱序, 或自行构造所需语义。

5.2 UDP 首部: 四个 16-bit 字段

UDP 首部固定 8 B:

Source Port | Destination Port | Length | Checksum

Length 统计 UDP header 与 data 的总字节数, 故最小值为 8; 已知 IP total length 与 IP header length 时, 也可反推 UDP length。Source Port 在 IPv4 经典规范中可为 0 表示未使用, 但 Destination Port 决定接收端服务入口。

Checksum 不只覆盖 UDP header 和 data, 还概念性覆盖来自 IP 的 pseudo-header: source/destination IP、上层 protocol/next-header 与 UDP length。伪首部不实际随 UDP 重复传输, 它把“这份数据应属于哪对网络端点、哪个上层协议”纳入完整性检查, 降低误投递未被发现的风险。IPv4 UDP checksum 可选与 IPv6 中通常必需的边界必须服从题设, 不能把 IPv4 的全零语义无条件搬到 IPv6。

6 TCP 首部: 连接状态在报文中的投影

TCP 基础首部最少 20 B, 可压缩为:

Src/Dst Port | Sequence Number | Acknowledgment Number
 Data Offset/Flags | Window | Checksum | Urgent Pointer
 Options+Padding | Data

字段应按“它暴露哪份状态”来读:

- port pair 与 IP pair 一起选择 connection;
- SEQ 标记本段首个数据字节, ACK 在 ACK flag 有效时表示下一期望字节;
- Data Offset 以 32-bit word 为单位给出 TCP header length, 最小为 5, 故最小首部 $5 \times 4 = 20$ B;
- Window 表示从 ACK number 开始、该发送者当前愿意接收的字节数, 是接收方向对反向发送者发布的流控状态;
- Checksum 覆盖 pseudo-header、TCP header 与 data, TCP 中不是可选项;
- Urgent Pointer 只有 URG 置位才有意义; Options 由 Data Offset 划定, MSS 是建连时常见选项而非固定基础字段。

408 常用 flags 可用状态事件复原: SYN 同步序号, ACK 使 acknowledgment 字段有效, FIN 表示本方向不再发送, RST 异常复位/拒绝, PSH 请求及时向应用推进, URG 使紧急指针有效。SYN 与 FIN 各占一个序号; 纯 ACK 若不带数据且无 SYN/FIN, 通常不消耗序号。字段存在不表示语义总生效, 例如 ACK number 只有 ACK flag 置位才应解释。

TCP/UDP checksum 的 pseudo-header 是跨层校验输入, 不是“传输层首部的一部分”, 也不由中间路由器作为 TCP/UDP 字段逐跳改写。NAT 改写 IP/port 后必须相应维护校验和, 正是因为这些值参与端点完整性约束。

7 TCP: 把不可靠 packet 网络解释成可靠 byte stream

TCP 向应用提供有序 byte stream, 而不是保留应用 write 的消息边界。每个传输方向都有独立序号空间和状态:

Application bytes → Byte sequence space → Segments → ACKed contiguous prefix → Receive byte stream

TCP sequence number 标记 segment 中第一个 data octet 的序号; ACK 通常表示“下一期望 byte”, 也就是此前连续 byte 已收到。SYN 和 FIN 也占用 sequence space, 这使控制事件能与数据一起被可靠排序。

8 为什么连接建立需要同步双方状态

建立 TCP connection 不是“测试网络通不通”，而是让双方确认：

- 对端 endpoint 存在且愿意通信；
- 双方 initial sequence number 已知；
- 旧连接的迟到 segment 不应被误认作当前 connection 数据；
- 双向发送/接收状态可以进入 ESTABLISHED。

典型三次握手：

1 Client	Server
2 CLOSED	LISTEN
3 SYN, seq=x ----->	
4	<----- SYN+ACK, seq=y, ack=x+1
5 ACK, ack=y+1 ----->	
6 ESTABLISHED	ESTABLISHED

第三次消息的关键不是“为了凑三次”，而是让 server 知道 client 已收到 server 的 SYN/ISN。两次消息只能让 client 知道双向可达，server 仍无法确认其 SYN 已被 client 接受。

8.1 状态比报文名更重要

主动打开常见轨迹：CLOSED → SYN-SENT → ESTABLISHED。

被动打开常见轨迹：CLOSED → LISTEN → SYN-RECEIVED → ESTABLISHED。

同时打开、半开连接、RST 和重传会产生其他分支。做题必须用“收到什么 segment 前处于什么 state”推演，而不是只背三行握手图。

9 TCP 数据传输的三套约束

发送方在任意时刻可发送多少数据，至少同时受：

1. **可靠性状态**：哪些 bytes 已发送、已确认、允许重传；
2. **flow control**：receiver advertised window *rwnd*；
3. **congestion control**：sender 的 *cwnd*。

教材常压缩为：

$$W_{usable} \leq \min(rwnd, cwnd) - \text{bytes in flight.}$$

rwnd 来自接收端 buffer/应用读取状态，保护 receiver；*cwnd* 来自 sender 对 network path 的拥塞推断，保护 network。二者数值都像“窗口”，但状态所有者、反馈来源和失败对象不同。

10 Flow control：接收端怎样表达“我还能接多少”

接收端将可用 buffer 空间通过 advertised window 反馈给发送端。应用读取数据会释放空间；新数据到达会占用空间。基本不变量是：发送方不能让未消费数据超过接收方声明的可接受范围。

若 receiver 通告 zero window, sender 暂停正常新数据发送, 但必须保留恢复探测机制, 否则 window update 丢失可能造成双方永久等待。

Flow control 不证明数据已可靠交付, 也不代表路径没有拥塞。它只约束 receiver capacity。

11 TCP 可靠性如何实例化通用机制

TCP 使用 byte sequence number、ACK、retransmission timer 和 duplicate suppression, 把可靠传输的机制用于 byte stream。

但 TCP 不能简单等同 GBN 或 SR:

- ACK 通常具有 cumulative semantics;
- receiver 可以缓存 out-of-order bytes;
- retransmission 可以由 timeout 或 duplicate ACK/SACK 等信号触发;
- 具体实现与扩展比教材中的纯 GBN/SR 状态机更复杂。

因此 GBN/SR 是理解窗口正确性和代价的模型, 不是给 TCP 贴唯一协议标签。

12 连接终止: 两个方向必须分别关闭

TCP 是 full-duplex。一个 endpoint 不再发送, 并不意味着它也不能继续接收。典型主动关闭:

```
1 A sends FIN
2 -> B ACKs: A->B direction closed after prior bytes
3 -> B may continue sending
4 -> B sends FIN when its direction finishes
5 -> A ACKs
6 -> active closer waits in TIME-WAIT before state disappears
```

四次报文来自两个方向的关闭意图可能不同时发生; 若 ACK 与 FIN 可合并, 报文数可能少于四个。不能把“四次挥手”当作固定物理定律。

12.1 TIME-WAIT 维护什么

TIME-WAIT 让 active closer 有机会重发最后 ACK, 并让旧 connection 的迟到 segment 在相同 end-point tuple 被复用前消失。它用暂时保留状态换取连接实例之间的不歧义。

13 TCP 状态的一生

每条 connection 至少维护:

- local/remote endpoint;
- send/receive initial and current sequence variables;
- send/receive window;
- retransmission and timing state;
- connection state;
- congestion-control state (由拥塞 Topic 解释)。

```

1 Create endpoint
2 -> Open / Listen
3 -> Synchronize sequence spaces
4 -> Transfer bytes in both directions
5 -> Retransmit / reorder / flow-control as events occur
6 -> Half-close or abort
7 -> Release state after safety interval

```

TCP 的代价不是只多 20-byte header，而是 endpoint 长期持有并更新这些 per-connection states。

14 408 协议流程：UDP/TCP 逐字段推进

14.1 UDP demultiplexing 与 checksum

发送应用产生一个 datagram；UDP 加入 source port、destination port、length 和 checksum 后交给 IP。接收端先按 IP protocol 找到 UDP，再验证 length/checksum，最后以 local address/port 等绑定状态选择 endpoint；无匹配 endpoint 时不能把 payload 随意交给某个 process。UDP checksum 检查通过只保护报文完整性，不产生重传、排序或连接状态。

14.2 三次握手：同步两个序号空间

步骤	segment	Client state / knowledge	Server state / knowledge
1	SYN, seq= x	CLOSED $\rightarrow$ SYN-SENT; 占用 x	LISTEN 收到 client ISN
2	SYN+ACK, seq= y , ack= $x + 1$	知道 server 收到 x 且获知 y	SYN-RECEIVED; 等待 y 被确认
3	ACK, ack= $y + 1$	ESTABLISHED	确认 client 已收到 server ISN; ESTABLISHED

SYN 重传、同时打开和 RST 是分支；正常三行不是所有输入下的唯一轨迹。每一步都必须依据收到的 flag、SEQ、ACK 和当前 state 判断是否合法。

14.3 数据、ACK、超时与快速重传接口

TCP SEQ 指向本 segment 首个 data octet；ACK 指向下一期望 byte。收到乱序 segment 时可缓存并重复发送累计 ACK；连续缺口被填上后 ACK 才跨过该段。RTO 到期触发可靠性重传，并向 NET07 提供强拥塞信号；多个 duplicate ACK 可触发 fast retransmit，并把拥塞窗口分支交给 NET07。这里 Owns TCP 报文与连接状态，NET03 Owns 通用正确性，NET07 Owns 'cwnd' 更新。

14.4 接收窗口与 zero-window probe

接收方通告 'rwnd = receive buffer capacity - buffered unread bytes' 的题设模型。发送方正常可用量为

$$\max\{0, \min(rwnd, cwnd) - FlightSize\}.$$

若收到 zero window，发送方暂停正常新数据，但保留 persist/probe 分支；探测促使接收方重报当前窗口，避免窗口更新丢失后双方永久等待。该机制保护 receiver，不证明 network 未拥塞。

14.5 关闭、半关闭、TIME-WAIT 与 RST

FIN 只关闭一个发送方向，并占用一个 sequence number；对端 ACK 后仍可在反方向发送。主动关闭方确认对端 FIN 后进入 TIME-WAIT，以便重发最终 ACK 并隔离旧 segment。RST 表示拒绝、异常终止或不存在的连接状态，不执行有序半关闭。题目若把 ACK 与 FIN 合并，报文数可以少于经典四段，但两个方向的关闭状态仍必须分别核对。

15 端点核验层：端口空间、绑定与分用

port number 是 16-bit 本地命名空间，范围为 0 ~ 65535。408 常用分区为：

范围	教材名称	正确边界
0 ~ 1023	熟知端口	公共服务的约定入口；不等于某个固定 process ID
1024 ~ 49151	登记端口	可登记的服务标识；仍只有本地主机语义
49152 ~ 65535	动态/短暂端口	常由客户端临时分配；“客户端”是使用惯例，不是协议身份

端口号不携带 OS process identity，也不跨主机形成全局对象。它回答“到达这台 host 后应选择哪个 transport endpoint”，而不是“远端操作系统此刻的 PID 是什么”。软件 port 与交换机/路由器的物理 interface 只是中文同名，Owner 和作用域完全不同。

在 408 默认模型中，UDP 分用主要依据 local destination address/port 与 protocol binding，同一 endpoint 可以接收来自不同 remote endpoint 的 datagram；TCP 则必须用 local/remote 两端的 IP 和 port 选择某条 connection state：

UDP： (protocol, local IP, local port),

TCP： (local IP, local port, remote IP, remote port).

真实 socket API 可通过 wildcard bind、connected UDP、reuse 与 namespace 规则进一步细化；若题设没有这些扩展，不应用实现细节覆盖教材判据。收到 UDP datagram 却不存在匹配端口时，主机通常丢弃并产生 ICMP Port Unreachable；该差错报文的网络层语义由 NET-04 拥有。

16 UDP 核验层：长度预算与反码校验

UDP Length 同时包含 8 B header 与 data。若 IPv4 没有选项，且 IP Total Length 为 L_{IP} ，则

$$L_{UDP} = L_{IP} - 20, \quad L_{data} = L_{UDP} - 8.$$

例如以太网 MTU 1500 B 且 IP header 为 20 B 时，不分片 UDP payload 上限为 $1500 - 20 - 8 = 1472$ B，而 UDP Length 字段为 1480。长度对象没先命名清楚，就会在 1472/1480/1500 三个数之间错位。

IPv4 普通最小首部下，受 IP Total Length 限制的 UDP payload 理论上限为

$$65535 - 20 - 8 = 65507 \text{ B},$$

但路径 MTU 往往给出更严格约束；IPv6、扩展首部和 jumbogram 等不套用这个 IPv4 教材算式。

UDP/TCP checksum 的教材计算都采用 16-bit one's-complement sum。以 UDP 为例：

1. 校验和字段先置零，拼接 pseudo-header、UDP header 和 data；
2. 按 16 bit 分组，奇数字节 payload 末尾只为计算补一个零字节，该字节不发送也不计入 UDP Length；

3. 每次溢出的最高位回卷加到低 16 bit，即 end-around carry；
4. 对最终和逐位取反，写入 checksum；接收端把收到的 checksum 一起求和，无错时结果为全 1。

IPv4 pseudo-header 包含 source/destination IP、零字节、protocol 17 和 UDP Length；TCP 使用 protocol 6 与 TCP segment length。它能把部分错主机、错协议和错长度纳入端到端完整性检查，但仍只是弱检错，不提供纠错、重传或认证。若 UDP 计算结果为全零，IPv4 在线字段需用全 1 表示该数值，以免与“未启用 checksum”的全零编码混淆；IPv6 不继承 IPv4 的可选校验语义。

17 TCP 首部与选项预算核验层

Data Offset 的 4-bit 数值以 4 B 为单位，所以 TCP header 为 20 ~ 60 B，options+padding 最多 40 B：

$$L_{header} = 4 \times \text{DataOffset}, \quad L_{options} = 4 \times \text{DataOffset} - 20.$$

因此 Data Offset 为 8 时，header 为 32 B、选项区为 12 B。任何 MSS、SACK 或 segment payload 计算都应先用这个字段划定真实边界。

MSS 是 TCP data 的方向性上限，不含 TCP/IP header；在无选项 IPv4 教材模型中常取

$$\text{MSS} = \text{MTU} - L_{IP\ header} - L_{TCP\ header}.$$

双方在 SYN 中各自**通知**自己愿意接收的 MSS，两个方向可不同；它不是把两个值讨价还价成一个共同值，也与当时 rwnd 数值不是同一对象。路径改变仍可能让原先不分片的选择失效。

选项	典型长度	状态作用
MSS	4 B	SYN 中通知本方向可接收的最大 data size
Window Scale	3 B	把 16-bit Window 解释为左移后的更大范围；在建连阶段建立语义
Timestamp	10 B	提供 RTT 样本关联，并用于 PAWS 区分序号回绕前后的 segment
SACK Permitted/SACK	可变	建连时许可；数据期报告非连续已收字节块
NOP/EOL	1 B	对齐或终止选项解析

选项预算是共享的。一个 SACK block 需要两个 32-bit 边界，即 8 B；SACK option 另有 2 B kind/length，因此在独占 40 B 预算时 $2 + 8k \leq 40$ 给出最多 4 blocks。若同时携带 timestamp 等选项，可报告的 blocks 会更少，不能把“4”写成所有报文段的无条件常数。

32-bit byte sequence number 会按模 2^{32} 回绕。以 bit rate R 连续发送时，绕回时间约为

$$T_{wrap} = \frac{2^{32} \times 8}{R}.$$

真正风险不是回绕本身，而是旧 segment 尚在网络中时出现相同 sequence。Timestamp/PAWS 为序号之外增加时间维度；题目仍须结合 MSL、速率与题设选项判断，不能只看到 32 bit 就断言永不歧义。

18 连接状态核验层：十一状态与四类计时器

三次握手与四段关闭只是正常路径的报文投影。完整状态应按“已经发了什么、正在等什么”理解：

阶段	主动路径	被动路径/分支
建立	CLOSED → SYN-SENT → ESTABLISHED	CLOSED → LISTEN → SYN-RECEIVED → ESTABLISHED
单方关闭	FIN-WAIT-1 → FIN-WAIT-2 → TIME-WAIT → CLOSED	CLOSE-WAIT → LAST-ACK → CLOSED
同时关闭	FIN-WAIT-1 → CLOSING → TIME-WAIT	双方都可能经历同一分支

CLOSE-WAIT 等的是本端应用完成关闭，不是继续等对方报文；FIN-WAIT-2 已确认本端 FIN，只等反方向 FIN；LAST-ACK 等最终确认；TIME-WAIT 等安全时间。两端同时打开时都从 SYN-SENT 接收对方 SYN，经 SYN-RECEIVED 后形成同一条 connection，而不是两条四元组相同的连接。

SYN flood 的资源压力来自 server 在最终 ACK 到达前已经维护半开状态。SYN cookie 等属于工程缓解，不改变第三次握手的知识闭环：server 必须获得“client 已确认 server ISN”的证据后，正常路径才进入 ESTABLISHED。

计时器	触发状态	防止的失败
Retransmission/ RTO	data/SYN/FIN 等可靠发送后等待 ACK	segment/ACK 丢失造成永久等待
Persist	收到 zero window	非零 window update 丢失后双方死锁
Keepalive	长期无活动的已建连接	对端崩溃留下永久占用的 connection state
TIME-WAIT (2MSL)	主动关闭方发最终 ACK 后	重发最终 ACK，并隔离旧 connection segment

2MSL 的两段最坏预算是：最后 ACK 最多存活一个 MSL 后丢失，对端因此重传的 FIN 最多再用一个 MSL 返回。若相同 endpoint tuple 的短连接受 TIME-WAIT 占用，而可用短暂端口数为 N ，教材化的同一对端新建速率上界约为 $N/(2MSL)$ ；真实系统还受源地址、端口分配与复用策略影响。

19 TCP 可靠性核验层：三个指针、RTO 与 SACK

本节不重新拥有 NET03 的通用滑动窗口证明，只拥有 TCP byte space 中的具体映射。发送端可用三个 byte pointers 表示窗口：

$$\underbrace{[\dots, P_1)}_{\text{已确认}} \underbrace{[P_1, P_2)}_{\text{已发未确认}} \underbrace{[P_2, P_3)}_{\text{可发送}} \underbrace{[P_3, \dots)}_{\text{暂不可发送}} .$$

因此

$$P_3 - P_1 = W_{\text{send}}, \quad P_2 - P_1 = \text{FlightSize}, \quad P_3 - P_2 = W_{\text{usable}}.$$

P_1 由 cumulative ACK 向前推动且不能后退； P_2 由本端实际发送推动； P_3 由最新有效窗口约束决定。窗口后沿滑动、前沿张缩和“已经发到哪里”是三件事，不能用一个窗口数替代全部状态。TCP 强烈不鼓

励窗口收缩，因为 sender 可能已经按旧前沿发出 data。

TCP 对 RTT 的自适应估计可按题设教材模型写成：

$$\begin{aligned} \text{SRTT} &\leftarrow (1 - \alpha)\text{SRTT} + \alpha R, \\ \text{RTTVAR} &\leftarrow (1 - \beta)\text{RTTVAR} + \beta|R - \text{SRTT}|, \\ \text{RTO} &= \text{SRTT} + 4\text{RTTVAR}. \end{aligned}$$

常用教材参数为 $\alpha = 1/8, \beta = 1/4$ ；第一样本常初始化 $\text{SRTT} = R, \text{RTTVAR} = R/2$ 。不同题目对本轮偏差使用更新前还是更新后的 SRTT 可能有口径差异，必须按题设顺序代入并在全过程保持一致。

重传后到达的 ACK 无法仅凭累计确认判断对应原发送还是重传，直接采样会把 RTT 估计拉长或缩短。Karn 原则因此不采纳重传 segment 的模糊 RTT sample，并在连续超时时对 RTO 做指数退避；获得未重传 segment 的无歧义样本后再恢复估计。Timestamp echo 能提供额外关联信息，但是否启用仍由连接选项状态决定。

SACK 不替代 cumulative ACK。若连续前缀到 1000，已额外收到 [1501,3001) 与 [3501,4501)，则 ACK 仍为 1001，SACK blocks 报告两个半开区间；右边界指向块后第一个 byte，而不是最后一个 byte。SACK 说明“接收端已有何块”，具体重传调度仍由 sender 算法决定。

20 流量控制核验层：零窗口、小报文与吞吐上界

接收端窗口是动态库存：data 到达增加 buffered unread bytes，应用读取减少它们。因此 rwnd 可增也可减，且不超过 receive buffer capacity。发送端当前可发新数据仍应计算

$$W_{usable} = \max\{0, \min(rwnd, cwnd) - \text{FlightSize}\}.$$

窗口值传播存在时延，所以 sender 当前保存的 rwnd 不等于 receiver 此刻内存中的瞬时空闲量；它是一份经报文通告的远端状态。

zero window 限制的是普通新数据，不等于丢弃一切 TCP segment。408 教材模型要求仍处理 ACK、zero-window probe 与携带 urgent data 的 segment。收到 $rwnd = 0$ 后启动 persist；若非零 update 丢失，probe 迫使 receiver 再次报告当前窗口。这个计时器解决 flow-control deadlock，不判断 peer 是否存活，也不替代 RTO。

小报文低效有两条不同因果链：

问题	First Divergence	控制思路
发送端 tiny-grams	应用逐字节写入，sender 立即逐段发送	Nagle：首段发出后缓存小写入，待 ACK 或积累到足够规模再发
Silly Window Syndrome	receiver 每腾出极小空间就通告极小窗口	Clark：空间达到一个 MSS 或缓存相当比例后再通告非零窗口

两者都以等待换 header efficiency，但 Owner 不同：Nagle 控 sender segmentation，Clark 控 receiver advertisement。低时延应用可能有不同策略，题目若明确禁用/修改算法应服从题设。

只考虑 flow control、且 sender 有持续数据时，窗口给出的理想上限约为 $rwnd/RTT$ ；长期稳态还受接收应用消费速率 r_{app} 、network capacity 与 congestion control 约束：

$$\text{Throughput} \leq \min \left\{ \frac{rwnd}{RTT}, r_{app}, \text{network/congestion limit} \right\}.$$

若应用永不读取，初始 buffer 填满后长期吞吐趋于 0；这说明 flow control 是 receiver capacity 的硬安全边界，而不只是“稍微慢一点”。

21 概念边界

概念 A	≠	概念 B	真正区别与题目信号	混淆后果
Port	≠	Process	port 是 namespace；process 使用 endpoint	认为一端口只能对应一次会话
Socket endpoint	≠	TCP connection	endpoint 是一端；connection 由两端共同确定	四元组与本地绑定混乱
UDP datagram	≠	TCP byte stream	UDP 保留 message boundary；TCP 保证 byte order	应用分帧假设错误
Connection establishment	≠	Data reliability	握手同步 state；可靠性需持续 ACK/重传	认为握手后不会丢包
ACK number	≠	Segment number	TCP ACK/SEQ 以 byte space 为核心	按 packet 个数推进序号
rwnd	≠	cwnd	receiver capacity 与 network capacity	流控/拥塞更新写反
FIN	≠	RST	有序关闭一个方向 vs 异常终止/拒绝	关闭状态机推演错误
TIME-WAIT	≠	server 固定等待	active closer 的最终 ACK 与旧段隔离	只按 client/server 身份判断
Pseudo-header	≠	Transmitted header	只参与 checksum，字段来自 IP	错算线上首部长度
UDP Length	≠	IP Total Length	UDP header+data vs 还含 IP header	反推 payload 漏减首部

22 做题调用协议

1. 写 transport protocol 和完整 endpoint tuple；
2. 明确应用需要 datagram 还是 byte stream 语义；
3. TCP 题为两个方向分别画 SEQ/ACK/state/window；
4. 先确定 SYN/FIN 是否占序号，再计算 ACK；
5. 窗口题分开 bytes ACKed、in flight、rwnd、cwnd；
6. 建连/关闭题逐事件更新 state，不背固定报文数；
7. 最后检查同一 server port 的不同 connections 是否被误合并。
8. 首部题先用 Data Offset/Length 划定边界，再按 flags 判断 SEQ、ACK、Window、Urgent Pointer 是否有效；
9. checksum 题把 pseudo-header 列为计算输入，但不得把它计入实际传输首部长度。

23 贯穿母例：同一 80 端口为什么能服务多个浏览器

server 在 203.0.113.8:80 listen。两个 clients 建立：

```
1 (192.0.2.10:51000, 203.0.113.8:80)
2 (198.51.100.20:52000, 203.0.113.8:80)
```

server local port 相同，但 remote IP/port 不同，因此是两条独立 connection，各自拥有 sequence、window、timer 和 state。若只看 destination port，会把多条状态机错误地合成一条。

24 高频 First Divergence

- 把 port 写成应用程序本身：没有区分 namespace 与 owner；
- TCP sequence 每 segment 加 1：忘记它以 byte 为基本坐标，SYN/FIN 另有占用；
- 三次握手解释为“第三次确认数据”：没有说明 server ISN 的确认闭环；
- 看到窗口只取 rwnd 或只取 cwnd：漏掉共同约束和 in-flight 数据；
- 把 TCP 判成 GBN：把教材可靠协议类比当成协议事实；
- 默认 client 总进入 TIME-WAIT：没有看谁主动关闭。
- 把 Data Offset 当字节数：忘记其单位是 32-bit word；
- 把 pseudo-header 画进在线 TCP/UDP 首部：混淆校验输入与传输字段；
- 看到 ACK number 就推进确认：没有先检查 ACK flag 与当前连接状态。

25 一页压缩与复原问题

IP host delivery → Port demultiplexing → Endpoint pair
→ Sequence-space synchronization → Bidirectional byte-state evolution

1. 为什么 port 不能唯一标识一条 TCP connection？
2. UDP 少掉的状态给应用带来什么自由和责任？
3. 三次握手的第三次究竟让谁知道了什么？
4. rwnd、cwnd 和 reliable sequence state 分别保护什么？
5. TIME-WAIT 怎样避免新旧 connection 混淆？
6. TCP/UDP 的 pseudo-header 为什么能发现一部分误投递，又为什么不增加线上首部长度？

26 来源与校正说明

- 早期归档中的《传输层-TCP》《传输层-传输层功能与 UDP》为空文件，因此旧版没有把空入口当作已有模型；本轮已另行核验新提供的新提供的中文补充笔记；
- transport/tcp-vs-udp.md 已吸收 transport functions、port ranges、UDP/TCP demultiplexing 与 connection state 对比；
- transport/udp.md 已吸收 UDP length budget、pseudo-header、反码求和、奇数字节填充与端口不可达接口；
- transport/tcp-basics.md 已吸收 header/options、MSS 方向性通知、sequence wrap、window

scale、timestamp/PAWS 与 option budget;

- `transport/tcp-connection.md` 已吸收完整 lifecycle、同时打开/关闭、2MSL、半开资源与四类 timers;
- `transport/tcp-reliable.md` 已按 NET03 边界吸收 TCP byte pointers、SRTT/RTO、Karn、SACK 与丢包判据接口;
- `transport/tcp-flow-control.md` 已吸收 rwnd inventory、zero-window exceptions/persist、Nagle、Clark 与吞吐上界; 其中 cwnd 更新仍由 NET07 拥有;
- TCP byte sequence、ACK、connection state 与接口边界依据 RFC 9293 校正;
- 本册保留 408 的握手、关闭、流控与端口模型, 同时把现代扩展与纯 GBN/SR 类比限制在明确边界内。